Day 4 학습 정리 — 2026-06-29 (진행 중)

> LLM(생성 AI) 연결 단계. 프롬프트 조립 → API 호출 → 프로필 반영. 이 프로젝트는 **Gemini API**(Google AI Studio)로 진행.

세션 3 — `prompt_service.py` (프롬프트 조립)

1. 프롬프트(prompt)란?

AI에게 보내는 **글 한 덩어리**(질문지/지시문). "너는 누구고, 이 정보를 보고, 이런 규칙으로, 이런 형식으로 답해". AI는 이 글을 읽고 답을 만든다. → 프롬프트가 좋아야 답이 좋다.

2. 이 파일이 하는 일

아직 AI를 부르지 않는다. **빈칸 채우기**만 한다.

```
템플릿  "이름: {name}"  +  데이터 name="유리"  →  "이름: 유리"
```

(엑셀 메일머지, 양식 빈칸 채우기와 같은 원리)

3. f-string — 문자열에 변수 끼우기

문자열 앞에 `f`를 붙이고 `{변수}` 자리에 값이 들어간다.

```
name = "유리"; age = 12
text = f"이름: {name}, 나이: {age}"   # → 이름: 유리, 나이: 12
```

여러 줄은 `"""..."""`(따옴표 3개)로 감싼다.

4. for 반복문 — 리스트를 글로 풀기

`records`는 기록이 여러 개 든 **리스트**. 그냥 `{records}`를 넣으면 객체 덩어리가 찍힌다. → 하나씩 꺼내서 글로 옮겨야 한다.

```
records_text = ""                    # ① 빈 종이에서 시작
for r in records:                    # ② 기록을 하나씩 r에 담아 반복 (5개면 5번)
    records_text += f"- {r.lesson_date}: 활동 {r.activity}\n"   # ③ 한 줄씩 이어붙임
```

- `r`은 기록 한 개 → `r.lesson_date`, `r.activity` 처럼 **점(.)으로 꺼냄** (아이 `child.name`과 같은 원리)
- `+=` = 기존 문자열 뒤에 더 붙이기 (누적)
- `\n` = 줄바꿈(엔터). 없으면 다 한 줄에 붙는다.

5. 내가 했던 실수 & 배운 점

- **들여쓰기 = 함수 소속.** `def` 아래 들여쓰기 된 줄만 함수 안. 맨 왼쪽에 붙이면 함수 밖으로 튕긴다.
- **`print` vs `return`:** `print`는 화면에 보여주고 버림. `return`은 값을 호출한 쪽에 건넨다. 다음 파일(llm_service.py)에 넘겨야 하니 `return`.
- **반복문 안에는 "기록 정보"만.** "~을 작성합니다" 같은 AI 지시문은 반복문이 아니라 프롬프트 템플릿(`f"""`)에 들어간다.
- `recent_records`, `today_record`, `rag_context`는 안 만든 변수 → 지금은 안 씀. `rag_context`는 Day 8(RAG) 거.

완성 코드 형태

```
def build_prompt(child, records):
    records_text = ""
    for r in records:
        records_text += f"- {r.lesson_date}: 활동 {r.activity}, 태도 {r.attitude},
강사메모 {r.teacher_note}\n"

    prompt = f"""[역할]
당신은 아동 줄넘기 수업 강사를 돕는 AI 피드백 코치입니다.

[아이 프로필]
- 이름: {child.name}
... (프로필 필드들) ...

[최근 수업 기록]
{records_text}
"""
    return prompt
```

세션 4 — `llm_service.py` (Gemini API 호출) ✅ 완료 (2026-06-30)

완성 코드 형태 (직접 조립함)

```
import os
from dotenv import load_dotenv
from google import genai

load_dotenv()                           # ① .env 읽기  ─┐
api_key = os.getenv("GEMINI_API_KEY")   #               │ 함수 밖 = 프로그램 켤 때 1번
만
client = genai.Client(api_key=api_key) # ② client 만들기 ─┘

def generate_feedback(prompt):
    response = client.models.generate_content(
        model="gemini-2.5-flash",       # ③ 호출 (주문)
        contents=prompt,
    )
    return response.text                # ④ 답에서 글만 꺼내 돌려줌
```

핵심 판단: `load_dotenv·client`는 **함수 밖**에 둠 → 호출할 때마다 .env 다시 읽거나 client 또 만들 필요 없음(낭비). 호출(`generate_content`)만 함수 안.

만나면 또 나올 에러 2개 (직접 해결함)

1. `ImportError: cannot import name 'genai'` → 가상환경(venv) 안 켜고 전역 파이썬으로 돌려서 패키지 못 찾음.
   - 해결: `.\.venv\Scripts\Activate.ps1` → 줄 앞에 `(kids-feedback)` 뜨면 켜진 것. (폴더명 `.venv` ≠ 표시명 `kids-feedback`)
   - 교훈: **돌리기 전에 venv 켜졌나 `(kids-feedback)` 확인.**
2. `429 RESOURCE_EXHAUSTED, limit: 0` → 코드 문제 아님. 키가 속한 프로젝트의 **무료 할당량이 0**.
   - 해결: 모델 `gemini-2.0-flash` → `gemini-2.5-flash`로 변경하니 됨. (안 되면 새 프로젝트로 키 재발급)
   - 교훈: 키 인증까지 됐는데 429면 코드 아니라 **할당량/모델 문제**.

세션 5 — 프로필 반영 + JSON 출력 ✅ 완료 (2026-06-30)

프롬프트 엔지니어링 = 규칙을 글로 시키기

프롬프트에 데이터만 넣으면 AI는 "알아서" 답한다. 원하는 동작을 시키려면 **지시문을 글로** 넣는다.

- `[작성 규칙]`: 성향 톤 반영 / memory_hint·caution_note는 학부모용에 노출 금지(강사용만) / 비교·진단 금지 / 도전적 목표.
- `[출력 형식]`: "아래 JSON으로만 출력" + 틀(next_goal/teacher_feedback/parent_feedback).
- **결과:** 같은 기록인데 유리(위축)는 다음목표 12개·부드러운 톤, 민수(승부욕)는 20개·도전 톤. → 규칙이 실제로 지켜짐(memory_hint가 parent_feedback에 안 샘).

f-string 안에서 진짜 중괄호 쓰기 ⚠️

`f"""..."""` 안에서 `{ }`는 변수 자리. **글자 그대로 `{`를 쓰려면 `{{`, `}`는 `}}`로 2개씩.** 안 그러면 SyntaxError로 import 자체가 안 됨. (JSON 예시 넣을 때 꼭 필요)

또 낸 실수 (전부 "이름 안 맞음" 류)

- `{recent_records}` / `{today_record}` — **안 만든 변수**를 씀 → 만든 변수는 `records_text`.
- `{r.records_text}` — `records_text`는 통짜 변수인데 `r.`(점)을 붙임. **점은 객체 속 꺼낼 때만**(child.name, r.activity). 통짜 변수는 점 없이.
- `return prompts` — 만든 이름은 `prompt`. **만든 이름 = 부르는 이름** 이어야 함(NameError).
- 교훈: 변수는 만든 철자 그대로. for문 변수 `r`은 루프 밖에서 쓰지 말기.

테스트 팁

DB 없이 빠르게 확인할 땐 `SimpleNamespace`로 가짜 객체를 만들어 점(.)으로 꺼내게 함. (`test_prompt.py`, 나중에 삭제 가능)

API / API 키 개념

- **API:** 남의 프로그램(여기선 Gemini)에 일을 시키고 결과를 받아오는 창구.
- **API 키:** 그 창구를 쓸 권한을 증명하는 **비밀번호**. → 코드에 직접 적거나 GitHub에 올리면 안 됨. `.env` 파일에 숨기고 `.gitignore`로 제외.

- 키 형식: 요즘 Google AI Studio 키는 `AQ.A...`로 시작 (옛날엔 `AIza...`였음). 둘 다 정상.

매개변수(parameter) — 함수가 "받아오는 재료"

함수를 **믹서기**로 생각. 그냥 못 돌리고 뭘 넣어줘야 갈린다.

```
def 믹서기(과일):          # '과일' = 매개변수 = 받는 재료를 담는 빈 그릇
    return 과일 + "주스"

믹서기("딸기")             # "딸기"가 '과일' 자리에 들어감 → "딸기주스"
믹서기("사과")             # 같은 함수인데 "사과주스" → 넣는 값에 따라 결과가 달라짐
```

- 매개변수는 **미리 정해진 값이 아니라 "부를 때 채워질 빈 그릇"**. 지금은 비어 있다.
- 함수를 **호출하는 쪽**이 값을 넣어준다.

우리 함수에 대입:

```
def generate_feedback(prompt):     # 'prompt' = 받는 글을 담을 빈 그릇
    ... contents=prompt ...         # 받은 그 글을 Gemini한테 보냄
```

`prompt`가 어디서 오냐 → 어제 만든 `build_prompt(child, records)`가 만든 글이 들어온다.

```
글덩어리 = build_prompt(child, records)   # 프롬프트 글 생성
generate_feedback(글덩어리)               # 그 글이 prompt 자리에 들어감
```

→ 그래서 `generate_feedback`을 쓸 땐 prompt가 어디서 오는지 신경 안 써도 됨. "누가 글을 넣어주면 Gemini한테 보낸다"만 하면 끝.

이미 써본 개념: 어제 `build_prompt(child, records)`의 `child`, `records`도 똑같은 매개변수.

한 줄 요약: 매개변수 = "부를 때 채워질 빈 그릇". 호출하는 쪽이 값을 넣는다.

`generate_feedback`을 왜 만드나 (역할)

- `prompt_service.py` = 아이 정보+기록을 **글(프롬프트)로 조립** (주문서 작성). AI는 아직 안 부름.
- `llm_service.py` = 그 글을 **실제로 Gemini에 보내고 답을 받아옴** (주방에 주문 넣고 음식 받기).
- `return` 하는 이유: 받은 피드백 글을 Day 5에서 **DB 저장**, Day 11에서 **화면 표시**로 넘겨야 함. `print`(찍고 버림) 아니라 `return`(값 전달).